

동시성을 위한 Go

2편: 코드 모델링 - 순차적 프로세스 전달

남궁명수

[02-1] 동시성과 병렬성의 근본적인 차이	3
(1) 문맥(Context)에 따른 논리적 고립	4
(2) 추상화 계층에 따른 동시성의 난이도	5
(3) Go 언어가 제시하는 새로운 추상화 링크	5
[02-2] CSP란?	7
(1) CSP의 탄생과 역사	7
(2) 수학적 모델로의 진화	7
(3) 입출력 명령과 Go 채널의 모태	8
(4) 결론: 동시성을 대중의 품으로 이끈 Go 언어	9
[02-3] 이것이 너에게 어떻게 도움이 되는지	10
(1) 하드웨어 고민에서 해방되다	10
(2) 웹 서버를 만들 때 생각의 차이	10
(3) 프레임워크 뒤에 숨겨진 복잡성의 진실	11
(4) Go 런타임이 주는 ‘공짜 점심’	12
(5) 미세한 단위의 동시성과 암달의 법칙	12
(6) 퍼즐을 완성하는 채널과 Select문	13
(7) 결론	13
[02-4] Go언어의 동시성 철학	14
(1) Go언어 공식 가이드가 주는 혼란	14
(2) 채널 vs 자물쇠 선택을 위한 4가지 나침반	15
(3) 기존의 구시대적 패턴 버리기	16
(4) 결론	16

[02-1] 동시성과 병렬성의 근본적인 차이

여러분 중에는 이 모든 컴퓨터 과학 역사와 이론이 흥미로울 수도 있고, 아닐 수도 있다.
하지만 이 책을 읽고
“동시성”과 “병렬성”의 차이는 많은 개발자들 사이에서 자주 무시되거나 오해받곤 한다.
수많은 대화 속에서 두 단어는 그저 “무언가가 다른 것과 동시에 똑같이 실행되는 것”이라는
뜻으로 혼용되곤 한다.

가끔은 병렬성이라는 단어가 맞을 때도 있지만, 대부분 개발자들이 ‘코드’에 관해
이야기할 때는 사실 ‘동시성’이라는 단어를 쓰는 것이 정확하다.

이 둘을 굳이 구별하려는 이유는 단순한 까다로운 척을 하려는 게 아니다.
동시성과 병렬성의 차이를 이해하는 것은 코드를 설계(모델링)할 때 엄청난 힘을 발휘하는
강력한 추상화 도구가 되며, Go언어는 이 장점을 100% 활용하고 있기 때문이다.

매우 심플하지만 뼈가 있는 한 문장으로 시작해보겠다.

가장 중요한 대전제

- 동시성은 ‘코드’가 가진 속성이다.
- 병렬성은 ‘실행 중인 프로그램(런타임)’이 가진 속성이다.

참 흥미로운 구분이다.
우리는 보통 이 두 가지를 똑같이 생각하곤 한다.
“병렬로 실행되라고 코드를 짜는 거 아닌가?”하고 말이다.

내가 프로그램의 두 덩어리가 병렬로(완전히 동시에 물리적으로) 돌아가기를 바라는 마음으로
코드를 짰다고 해서, 프로그램이 실행될 때 실제로 그렇게 돌아간다는 보장이 있을까?

만약 그 코드를 ‘두뇌(코어)가 딱 1개인 컴퓨터’에서 실행하면 어떻게 될까?

어떤 분들은 “그래도 병렬로 돌겠죠!”라고 생각하겠지만, 사실은 그렇지 않다.
프로그램의 파편들이 겹보기에는 동시에(병렬로) 도는 것처럼 보일지 몰라도,
실제로는 컴퓨터가 구별할 수 없을 정도로 엄청나게 빠르게 순서대로(순차적으로) 번갈아
가며 실행하는 것뿐이다.

CPU가 여러 프로그램 사이에서 자리를 바꾸는 ‘문맥 전환(Context Switch)’을 하며 시간을
쪼개 쓰는 것이다. 충분히 긴 시간의 돋보기를 대고 보며 마치 동시에 도는 것처럼 보일 뿐이
다.

하지만 이 똑같은 프로그램 파일을 ‘코어가 2개인 컴퓨터’에서 실행하면,
그제야 비로소 두 프로그램 덩어리가 진짜로 ‘병렬’하게 각자의 두뇌를 쓰고 동시에 실행될 수
있다.

이 사실은 우리에게 아주 흥미롭고 중요한 진실들을 밝혀준다.

1. 우리는 ‘병렬 코드’를 짤 수 없다. 우리는 그저 ‘병렬로 실행되기를 희망하는 동시성 코드’
를 작성할 수 있을 뿐이다. 다시 말하지만, 병렬성은 코드가 아니라 내 프로그램이 돌아가는
환경(런타임)이 결정한다.

2. 내가 짠 동시성 코드가 실제로 병렬로 돌고 있는지 아닌지 전혀 몰라도(무지해도) 괜찮으며, 오히려 모르는게 더 바람직하다. 이것이 가능한 이유는 우리 프로그램 밑바닥에 수많은 추상화 계층(동시성 도구들, 프로그램 런타임, 운영체제, 도커/가상머신 같은 플랫폼, 그리고 최종 하드웨어 CPU)이 단단히 버티고 있기 때문이다.

이 든든한 추상화 덕분에 우리는 동시성과 병렬성을 분리해서 생각할 수 있고, 코드를 자유롭게 유연하게 표현할 수 있게 된다.

3. 병렬성은 ‘시간’ 혹은 ‘문맥’의 함수이다. 1장의 ‘원자성’ 파트에서 문맥을 ‘어떤 행동이 원자적인지 판단하는 범위’로 정의했던 것을 기억하나? 여기 병렬성에서의 문맥은 ‘두 개 이상의 작업이 병렬인지 판단하는 범위’로 정의된다.

예를 들어, 우리의 판단 범위(문맥)가 ‘5초’라는 넉넉한 시간이고, 각각 1초씩 걸리는 작업 두 개를 실행했다면 우리는 이 두 작업이 대략 같은 시간대에 실행되었으므로 “병렬로 돌았다”고 말할 수 있다. 하지만 판단 범위가 딱 ‘1초’로 아주 미세하다면, 두 작업은 겹치지 못하고 “순서대로(순차적으로) 돌았다”고 판단해야 할 것이다.

(1) 문맥(Context)에 따른 논리적 고립

방금 설명한 시간적 문맥 이야기가 너무 추상적일 수 있으니, 현실적인 예시를 들어보겠다. 이번엔 우리가 설정할 문맥(범위)은 ‘여러분의 컴퓨터’이다.

현대 물리학적인 상상을 배제한다면, 내 컴퓨터에서 실행 중인 어떤 프로그램(프로세스)이 여러분의 컴퓨터에서 실행 중인 프로그램의 논리에 영향을 주지 않을 거라고 생각하는 것은 매우 당연하다. 우리가 각자의 컴퓨터에서 계산기 프로그램을 켜고 더하기 빼기를 한다고 해서, 내가 누른 버튼이 여러분의 계산기 결과에 영향을 주지 않는다.

참 뻔하고 당연한 이야기지만, 이 안에는 모든 요소가 들어가 있다. 여기서 ‘컴퓨터 기기 자체’가 문맥이 되고 ‘계산기 프로그램’이 동시성 작업이 된다.

우리는 세상을 독립된 컴퓨터, 독립된 운영체제(OS), 독립된 프로세스라는 추상화 단계로 나누어 바라보기 때문에, “내 계산기는 안전하고 정확하다”라는 확신을 가질 수 있는 것이다.

이게 정말 당연하고 뻔한 예시일까?

각자 개인용 컴퓨터(PC)를 한 대씩 가지고 있는 것이 지금은 당연해보이지만, 1970년대 후반까지만 해도 거대한 공용 컴퓨터(메인 프레임) 한 대를 온 회사가 나눠 쓰는 것이 당연했다. 그 시절 개발자들이 “동시성”을 고민할 때의 문맥(범위)은 컴퓨터 기기가 아니라 하나의 컴퓨터 안에서 돌아가는 ‘프로그램의 프로세스’ 단위였다.

재미있게도, 수많은 개발자가 ‘분산 시스템’이나 클라우드를 다루는 지금에 와서는 이 문맥이 다시 과거처럼 확장되고 있다. 이제 우리는 가상화 플랫폼(하이퍼바이저), 컨테이너(도커), 가상 머신(VM) 자체를 하나의 거대한 동시성 문맥으로 바라보기 시작했다.

(2) 추상화 계층에 따른 동시성의 난이도

내 컴퓨터의 프로세스가 다른 사람 컴퓨터의 프로세스에 영향을 주지 않는 것은 확실하다(같은 분산 시스템으로 묶여있지 않다면).

그렇다면 범위를 좁혀서, '동일한 한 컴퓨터 안에서 돌고 있는 두 개의 프로세스'는 서로에게 전혀 영향을 주지 않을까?

프로세스 A가 몰래 파일을 덮어써서 프로세스B가 엉뚱한 파일을 읽게 만들 수도 있고, 보안이 취약한 운영체제(OS)라면 프로세스 A가 프로세스 B의 메모리를 오염시킬 수도 있다. 해커들이 쓰는 많은 기법이 실제로 이런 방식으로 작동한다.

그럼에도 불구하고, '프로세스' 수준까지는 머릿속으로 코드를 설계하기가 비교적 쉽다. 한 컴퓨터에서 두 사용자가 각자 계산기 프로세스를 켜고 놀아도, OS가 프로세스 벽을 단단히 쳐주기 때문에 서로 완벽히 격리되어 있다고 안심할 수 있다.

하지만 개발자가 한 단계 더 깊은 밑바닥 세계로 내려가는 순간, 동시성의 짐은 무겁게 어깨를 짓누르기 시작한다.

범위를 한 단계 더 아래인 '운영체제 스레드(OS Thread)의 경계'로 좁히면 어떻게 될까?

바로 우리가 1장에서 침을 튀기며 배웠던 그 악명 높은 문제들이 떼거지로 몰려온다. 데이터 경쟁, 데드락, 라이브락, 기아 상태가 전부 이 스레드 수준에서 터지는 일들이다.

만약 모든 사용자가 단 하나의 계산기 프로세스를 공유하고, 그 안에서 여러 스레드가 동시에 돌아가는 시스템을 짜야 한다면 어떨까?

코드를 올바르게 작성하기가 수만 배는 더 힘들어질 것이다. 메모리에 자물쇠(Lock)을 언제 잠그고 풀지, 이 결과물이 원래 요청한 유저의 것이 맞는지 사망팔방 신경 써야 한다.

즉, 우리가 추상화 계층의 밑바닥(Stack)으로 내려가면 내려갈수록, 동시성 코드를 설계하고 이해하는 것은 극악으로 어려워지지만, 그만큼 시스템 성능과 정교함을 위해서는 아주 중요해진다. 역설적으로, 동시성을 올바르게 짜는게 너무나도 어렵기 때문에, 우리는 조립하기 쉽고 안전한 '동시성 도구(기본형)'가 간절하게 필요해지는 것이다.

그러나 안타깝게도, 우리 IT 업계의 대부분의 동시성 코드는 가장 원초적이고 다루기 힘든 계층 중 하나인 'OS 스레드' 수준에서 작성되어 왔다.

(3) Go 언어가 제시하는 새로운 추상화 링크

Go 언어가 세상에 공개되기 전까지, 자바나 C++같은 대부분의 유명한 프로그래밍 언어들의 추상화 체인은 'OS 스레드'에서 끝이 났다. 만약 동시성 코드를 짜고 싶다면 개발자가 직접 스레드를 설계하고, 그 스레드들 사이에 자물쇠(Mutex)를 채워가며 메모리 동기화를 제어해야 했다. 만약 동시 처리해야 할 작업이 너무 많아서 컴퓨터가 스레드 무게를 버티지 못하면, 직접 '스레드 풀'을 만들고 그 위로 작업을 분배(Multiplexing)하는 복잡한 코드를 구현해야 했다.

하지만 Go언어는 이 거칠고 위험한 추상화 체인 바로 밑에 하나의 견고한 링크를 새로 연결했다. 그것이 바로 고루틴이다.

더 나아가 Go언어는 컴퓨터 과학의 거장 토니 호어 박사의 연구에서 수많은 영감을 얻어, 우리에게 채널이라는 완전히 새로운 동시성 도구를 쥐여주었다.

우리가 지금까지 해온 생각대로라면, OS 스레드 밑에 또 다른 추상화 계층(고루틴)을 추가했으니 코드가 더 복잡하고 어려워져야 정상일 것 같다.

하지만 정말 신기하게도, Go에서는 동시성이 오히려 훨씬 쉬워진다.

왜냐하면 고루틴은 OS 스레드 위에 쌓아 올린 짐이 아니라, OS 스레드의 복잡함을 완전히 대체해 버린 도구이기 때문이다.

물론 내부적으로는 여전히 OS스레드가 바쁘게 돌고 있다. 하지만 우리 개발자들은 더 이상 머릿속으로 ‘OS 스레드’를 계산하며 고통바지 않아도 된다. 대신 우리는 오직 고루틴과 채널, 그리고 아주 가끔씩만 공유 메모리를 신경 쓰며 명쾌하게 문제를 모델링하면 된다.

이러한 혁신이 우리에게 얼마나 놀라운 편리함을 가져다주는지는 잠시 후에 알아보고, 먼저 Go언어가 이 위대한 아이디어들을 어디서 빌려왔는지, Go 동시성 도구의 뿌리가 된 토니 호어 박사의 기념비적인 논문 “순차적 프로세스 간의 통시(Communicating Sequential Processes, CSP)”의 세계로 떠나보겠다!

[02-2] CSP란?

(1) CSP의 탄생과 역사

CSP는 ‘순차적 프로세스 간의 통신(Communicating Sequential Processes)’의 약자로, 동시성을 다루는 기술적인 방법론이자 이 개념을 세상에 처음 제안한 논문의 이름이기도 하다.

1978년, 컴퓨터 과학의 거장 토니 호어 박사는 ACM(미국 컴퓨터 협회) 학술지에 이 기념비적인 논문을 발표했다.

그는 이 논문에서 “입력(Input)과 출력(Output)이야말로 프로그래밍 언어의 가장 핵심적인 기본 도구(Primitive)로 다뤄줘야 한다”고 주장했다. 특히 동시에 실행되는 동시성 코드에서는 더더욱 그렇다고 강조했다.

논문이 쓰이던 1978년 당시는 프로그램을 어떻게 구조화 할 것인가에 대한 연구가 활발하던 시기였다. 하지만 대부분의 노력은 순서대로 실행되는 ‘순차적 코드’에만 집중되어 있었다. “코드에 goto 문을 쓰는게 맞냐 틀리냐”로 싸우고 있었고, ‘객체 지향(OOP)’ 패러다임이 막 싹을 틔우던 시절이었다.

반면, 동시에 실행되는 ‘동시성 작업’에 대해서는 아무도 깊게 고민하지 않았다. 토니 호어 박사는 이 뒤쳐진 영역을 바로잡기 위해 연구를 시작했고, 그렇게 CSP가 탄생했다.

(2) 수학적 모델로의 진화

재미있는 사실은, 1978년 첫 논문에 등장한 CSP는 실제 컴퓨터 언어가 아니라, 오직 자신의 아이디어를 증명하기 위해 가상으로 만든 아주 단순한 의사 언어였다는 점이다. 토니 호어 박사 본인도 논문에 이렇게 적었다.

토니 호어 박사의 고백(1978)

“이 논문에서 소개하는 개념과 표기법들은 ... 추상적인 코딩이든 구체적인 코딩이든, 실제 프로그래밍 언어로 직접 사용하기에는 적절하지 않습니다.”

그는 자신이 제시한 기술이 프로그램의 정답을 완벽하게 검증해주지 못할까 봐 깊이 우려했고, 이 방식을 진짜 프로그래밍 언어에 구현했을 때 과연 좋은 성능을 낼 수 있을지도 확신하지 못했다.

하지만 그 후 6년 동안 CSP의 아이디어는 끊임없이 다듬어졌고, 마침내 ‘프로세스 대수’라고 불리는 컴퓨터 과학의 정식 수학적 모델로 발전하게 된다.

프로세스 대수란 동시성 시스템을 수학 공식처럼 모델링하고, 대수학 법칙을 이용해 이 시스템이 얼마나 효율적이고, 수학적으로 올바른지 검증하는 학문이다.

수학적 증명은 이 책의 범위를 벗어나므로 다루지 않겠지만, 분명한 것은 이 1978년의 CSP 원본 논문과 그 사상이 오늘날 Go 언어 동시성 모델의 가장 거대하고 직접적인 영감의 원천이 되었다는 점이다.

(3) 입출력 명령과 Go 채널의 모태

호어 박사는 입출력이 프로그래밍 언어 자체의 기본 도구가 되어야 한다는 자신의 주장을 증명하기 위해, 가상 언어에 프로세스 간의 통신을 담당하는 아주 직관적인 명령어를 넣었다.

그는 입력이 들어와야 실행되고, 그 출력을 다른 곳에서 가져다 쓰는 독립된 코드 덩어리들을 ‘프로세스’라고 불렀다.(그 시절에 프로그램 구조 논쟁이 없었다면 그냥 ‘함수’라는 단어를 썼을 것이다.)

그리고 이 프로세스들이 서로 대화(통신)할 수 있도록 다음과 같은 기호들을 만들었다.

- ! : 프로세스에 데이터를 보낼 때 쓰는(Send) 명령
- ? : 프로세스로부터 데이터를 읽어올 때 쓰는 입력(Receive) 명령

이 명령어들은 데이터를 어디서 읽어와서 어떤 변수에 넣을지, 혹은 어느 목적지 프로세스로 보낼지 명확하게 지정해야 했다. 두 프로세스가 서로 딱 맞아떨어지면, 한 프로세스의 출력이 다른 프로세스의 입력으로 막힘없이 다이렉트로 흘러 들어갔다. 논문에 나온 예시들을 쉽게 살펴보겠다.

명령어 표현	부연 설명 (쉽게 풀이)
cardreader?cardimage	cardreader라는 장치(프로세스)로부터 데이터를 읽어와서 cardimage라는 변수에 저장해라.
lineprinter!lineimage	lineprinter라는 인쇄 장치로 lineimage 변수의 값을 보내 출력해라.
X?(x, y)	X라는 프로세스로부터 두 개의 값을 쌍으로 읽어와서 각각 x와 y 변수에 나누어 담아라.
DIV!(3*a+b, 13)	DIV라는 연산 프로세스로 지정된 두 개의 계산 값을 던져줘라.
*[c:character; west?c → east!c]	west 프로세스가 주는 문자들을 하나씩 읽어서 변수 c에 담은 뒤, 곧바로 east 프로세스로 한 땀 한 땀 토스해라. west 프로세스가 종료될 때까지 이 행동을 무한 반복한다.

Go 언어의 채널(chan)과 소름 돋을 정도로 닮았다.

특히 마지막 예시를 보면, west에서 나온 출력이 변수 c를 거쳐 곧바로 east로 흘러 들어간다. 두 프로세스가 완벽하게 동기화되어 매칭된 것이다.

다만 초기 CSP 논문에서는 통신할 때 반드시 대상 프로세스의 ‘이름’을 명확히 지정해야만 했다. 호어 박사도 이 방식이 라이브러리 코드를 짜서 남에게 나눠줄 때 문제가 될 수 있음을 직감했다. 내 코드를 가져다 쓰는 사람이 입력과 출력 프로세스의 이름까지 다 알아야 하니까.

그래서 그는 나중에 파라미터처럼 이름을 유연하게 갈아 끼울 수 있는 ‘포트 이름’ 등록 방식을 슬쩍 제안하기도 했는데, 이는 오늘날 우리가 사용하는 매개변수나 반환값과 매우 유사하다.

또한, 이 가상 언어는 1974년 에츠허르 데이크스트라 박사가 제안했던, ‘가드된 명령’이라는 개념을 도입했다. 문장 중간에 화살표(->)를 두고 왼쪽에 ‘조건’,

오른쪽에 ‘실행할 코드’를 두는 방식이다. 왼쪽 조건이 거짓이거나 이미 끝난 상태라면 오른쪽 코드는 절대로 실행되지 않는 규칙이었다. 이 가드된 명령이 호어 박사의 입출력 기호와 결합하면서, 오늘날 Go 언어의 select 문과 채널의 단단한 초석이 되었다.

(4) 결론: 동시성을 대중의 품으로 이끈 Go 언어

역사는 토니 호어 박사의 제안이 백 번 옳았음을 증명해 주었다.

하지만 흥미롭게도, Go 언어가 2000년대 후반에 출시되기 전까지 프로그래밍 역사상 그 어떤 메이저 언어도 이 CSP의 입출력 도구를 언어의 핵심 심장부(내장 기능)로 채택하지 않았다. 자바, C++, C#같은 대부분의 유명한 언어들은 CSP의 ‘메시지 패싱’ 스타일 대신, 1장에서 배운 ‘메모리를 공유하고 자물쇠(Mutex)로 동기화하는 방식’을 훨씬 선호하고 발전시켜 왔다. CSP 계열을 쓴 예외적인 언어들도 있었지만, 아쉽게도 대중적인 선택을 받지 못하고, 매니아 층에만 머물렀다.

Go 언어는 CSP의 위대한 원칙들을 언어의 코어(Core)에 완벽하게 이식하여, 이 강력하고 안전한 동시성 프로그래밍 스타일을 전 세계 대중들에게 보급한 최초의 메이저 언어이다.

Go가 이 방식으로 엄청난 대성공을 거두자, 이제는 다른 오래된 언어들도 부랴부랴 이 채널과 같은 도구를 추가하려고 열심히 뒤따라오고 있는 상황이다.

그렇다고 기존의 ‘메모리 접근 동기화(자물쇠 방식)’가 무조건 나쁘고 쓰지 말아야 할 구시대의 유물이라는 뜻은 아니다. 뒤에서 다루겠지만(Go의 동시성 철학), Go언어 안에서도 상황에 따라 자물쇠를 채우고 메모리를 직접 공유하는게 훨씬 효율적이고 적절할 때가 있다.

다만, 메모리를 공유하는 모델은 프로그램의 덩치가 커지고 복잡해질수록 사람이 머릿속으로 버그 없이 통제하기가 불가능에 가까워진다.

반면, Go언어는 처음 설계될 때부터 뼈대 속에 CSP의 원칙을 깊게 새겨두었기 때문에, 코드를 읽기도 쉽고, 쓰기도 편하며, 동시성 시스템을 인간의 뇌로 명쾌하게 이해할 수 있는 것이다. 이것이 바로 수많은 개발자가 Go 언어의 동시성을 가장 큰 축복이자 강점으로 꼽는 진짜 이유이다.

[02-3] 이것이 너에게 어떻게 도움이 되는지

(1) 하드웨어 고민에서 해방되다

여러분 중에는 이 모든 컴퓨터 과학 역사와 이론이 흥미로울 수도 있고, 아닐 수도 있다. 하지만 이 책을 읽고 있다면 여러분은 당장 해결해야 할 현실의 비즈니스 문제가 있을 것이고, “도대체 왜 Go 언어가 동시성에서 이토록 특별하다는거지?”라는 의문이 들것이다.

앞서 ‘동시성과 병행성의 차이’에서 다루었듯이, 기존의 많은 프로그래밍 언어들은 추상화의 단계를 ‘OS 스레드’와 ‘자물쇠(Mutex)’를 통한 메모리 동기화 수준에서 멈췄다. 반면, Go 언어는 이 거칠고 무거운 방식을 과감히 밀어내고 고루틴과 채널이라는 완전히 새로운 개념을 그 자리에 채워넣었다.

두 모델의 개념을 아주 단순하게 1:1로 비교해보면 다음과 같다.
(완벽히 똑같진 않지만 감을 잡는 데 도움을 줄 것이다)

- 고루틴 ~ 운영체제의 스레드
- 채널 ~ 자물쇠

그렇다면 이 새로운 추상화가 우리에게 구체적으로 무엇을 해줄까?

고루틴은 우리를 ‘병렬성(하드웨어 두뇌를 어떻게 나눌지)’이라는 복잡한 물리적 고민에서 해방시켜 준다. 그리고 문제가 가진 ‘자연스러운 동시성의 수준’ 그대로 코드를 짤 수 있게 돕는다.

이게 무슨 말인지 와닿지 않으실 테니, 바로 실무 예시로 들어간다.

(2) 웹 서버를 만들 때 생각의 차이

내가 사용자의 요청을 받아서 처리하는 웹 서버를 만들어야 한다고 가정해 보자.

스레드 중심 언어로 개발할 때의 머릿속

어떤 프레임워크도 쓰지 않고 오직 ‘스레드’만 제공하는 언어로 이 문제를 풀려고 하면, 개발자는 비즈니스 로직을 짜기도 전에 다음과 같은 기술적인 질문 공세에 시달리며 머리를 싸매야 한다.

- “내가 쓰는 언어가 스레드를 기본으로 지원하나? 아니면 별도 라이브러리를 골라야 하나?”
 - “스레드 간의 안전한 경계선을 어디에 쳐야 하지?”
 - “내가 서비스를 올릴 이 운영체제에서 스레드 하나를 생성하는 비용이 얼마나 무겁지?”
 - “리눅스 서버랑 윈도우 서버가 스레드를 다루는 방식이 다르면 어떡하지?”
 - “스레드가 너무 많이 켜지면 서버가 터지니까 ‘스레드 풀’을 만들어 제한해야겠다.
- 그런데... 우리 시스템에 딱 맞는 최적의 스레드 개수는 도대체 몇 개지?”

전부 중요한 질문들이다. 하지만 치명적인 문제가 있다.

이 질문 중 그 어떤 것도 내가 진짜 해결해야 하는 ‘웹 서버 비즈니스 기능’과는 아무런 상관이 없다는 점이다.

개발자는 코드를 시작하자마자 하드웨어와 병렬성을 어떻게 제어할 것인가라는 늪에 빠져버리게 된다.

Go 언어로 개발할 때의 머릿속

이제 한 걸음 물러나서, 우리가 해결하려는 현실의 문제를 아주 담백하고 자연스럽게 말로 표현하겠다.

“개별 사용자들이 우리 서버에 접속해서 세션(Session)을 연다.
세션은 사용자의 요청을 받아서 처리한 뒤 응답을 돌려준다.”

놀랍게도 Go 언어에서는 우리가 말한 이 자연스러운 상태 그대로 코드를 작성할 수 있다.
새로운 사용자가 접속하나요?

그럼 그냥 그 사용자를 위한 고루틴을 하나 커세요(`go handle()`).

그리고 그 안에서 요청을 처리하고 (필요하면 다른 고루틴과 채널로 대화하며),

처리가 끝나면 함수를 종료해서 고루틴을 닫으면 끝이다.

우리가 머릿속으로 문제를 생각하는 방식과 Go언어로 코딩하는 방식이 1:1로 완벽하게 일치하게 되는 것이다.

이것이 가능한 이유는 Go언어가 우리에게 강력한 약속을 하나 해주기 때문이다.

“고루틴이 상상 이상으로 가볍고 강력하니, 개수 제한 같은 건 처음에 전혀 걱정하지말고 그냥 켜고 싶을 때 켜라!”는 약속이다.

물론 시스템이 켜지면 고루틴 개수를 모니터링해야 할 때가 오지만, 시작도 하기 전부터 그걸 고민하는 것은 명백한 ‘조기 최적화의 오류’이다.

스레드를 쓸 때는 시작부터 개수 제한을 고민해야 하는 것과 완전히 대조적이다.

(3) 프레임워크 뒤에 숨겨진 복잡성의 진실

어떤 분들은 이렇게 반론할 수 있다.

“다른 언어도 좋은 프레임워크 쓰면 하드웨어 스레드 고민 안하게 다 숨겨주는데요?”

하지만 복잡성 병렬성 고민을 프레임워크가 숨겨준다고 해서 그 복잡성 자체가 사라지는 것은 아니다. 누군가는 그 무시무시한 프레임워크를 밤새워 짜야 했고, 여러분의 코드는 그 저자가 마주했던 수많은 복잡성의 타워 위에 아슬아슬하게 얹혀 있게 된다. 복잡성이 눈에 보이지 않는다고 해서 없는 게 아니며, 숨겨진 복잡성은 언제나 예상치 못한 버그를 낳는다.

반면 Go언어는 언어 자체가 태어날 때부터 ‘동시성’을 중심에 두고 설계되었다.

도구와 언어가 겹돌지 않고 완벽한 일체감을 이룬다. 마찰이 적으니, 당연히 버그도 훨씬 적게 생긴다.

(4) Go 런타임이 주는 ‘공짜 점심’

문제를 자연스럽게 코딩할 수 있다는 것 외에도, Go 언어의 구조는 엄청난 부가 이득을 가져다 준다.

Go 런타임은 고루틴을 OS 스레드 위로 자동으로 나누어 얹고 스케줄링을 대신 관리해준다. 이것이 대단한 이유는, 우리가 짠 코드는 단 한 줄도 고치지 않았는데, Go 개발진이 런타임을 업그레이드해 주면 우리 프로그램이 알아서 공짜로 빨라지기 때문이다. 완벽한 ‘관심사 분리’의 이점이다.

실제로 Go의 릴리즈 노트를 가만히 살펴보면 이런 문구들이 종종 등장한다.

“Go 1.5버전부터 고루틴을 스케줄링하는 내부 알고리즘 순서가 더 효율적으로 변경되었습니다.”

여러분은 가만히 앉아서 Go 버전만 올렸을 뿐인데, 보이지 않는 곳에서 엔지니어들이 내 프로그램을 더 빠르게 만들어 준 것이다.

또한, Go 런타임은 똑똑한 감시자 역할을 한다. 만약 어떤 고루틴이 파일 읽거나 네트워크 입력(I/O)을 기다리느라 멍하니 멈춰 서 있으면, 런타임이 이를 알아채고 그 무거운 OS 스레드를 쏙 빼앗아다가 지금 당장 일 할 수 있는 다른 고루틴에게 빛의 속도로 배정해준다. 이 덕분에 전체적인 코드 성능이 극적으로 향상된다.

(5) 미세한 단위의 동시성과 암달의 법칙

문제를 자연스럽게 동시성 코드로 옮길 수 있게 되면서 또 다른 나비효과는, 우리가 훨씬 더 촘촘하고 정교한 단위로 동시성 코드를 짜게 된다는 점이다.

현실 세계의 문제들은 대부분 동시 다발적으로 일어난다. 다른 언어였다면 스레드가 무서워서 ‘스레드 풀’에 사용자를 뭉뚱그려 집어 넣었겠지만, Go 언어에서는 모든 개별 사용자마다 고루틴을 하나씩 정직하게 할당해 줄 수 있다.

이렇게 동시성을 아주 잘게 쪼개어 짜놓은 프로그램은 성능 좋은 하드웨어(CPU 코어가 많은 서버)에 가져다 심는 순간 그 수많은 코어를 남김없이 백분 활용하여 다이내믹하게 성능이 확장된다. 우리가 앞에서 배웠던 ‘암달의 법칙’이 우리 편이 되어 강력하게 작동하는 순간이다.

(6) 퍼즐을 완성하는 채널과 Select문

가볍고 강력한 고루틴은 동시성 퍼즐의 한 조각일 뿐이다. CSP이론에서 빌려온 또 다른 무기인 채널과 select 문이 결합해야 비로소 마법이 완성된다.

- 채널의 결합성

채널은 다른 채널들과 부드럽게 결합할 수 있는 성질을 가진다.
여러 거대한 하위 시스템들에서 나오는 출력 채널들을 하나로 부드럽게 엮어서 제어할 수 있기 때문에 거대 시스템을 만들기가 아주 단순해진다.
채널에 타임아웃을 섞거나, 취소 신호를 결합하는 것도 너무나 쉽다.
수많은 자물쇠(Mutex)들을 서로 안 꼬이게 조율하는 끔찍한 작업과는 차원이 다르다.

- select 문의 강력함

select문은 채널의 단짝 친구이자, 채널 조립을 가능하게 만드는 핵심 열쇠이다.
select 문을 사용하면 어떤 채널에서 이벤트가 발생하는지 효율적으로 대기할 수 있고,
여러 채널에서 동시에 데이터가 쏟아질 때 한쪽만 편애하지 않고
공평하게 무작위로 메시지를 선택해 처리할 수 있으며, 기다리는 메시지가 없으면 그냥 하던 일 하러 지나가게 만들 수도 있다.

(7) 결론

토니 호어 박사의 CSP이론에서 영감을 받은 이 아름다운 동시성 도구들과,
이를 눈부시게 받쳐주는 고성능 런타임의 하모니가 바로 Go 언어를 움직이는 진짜 성장엔진이다.

1장과 2장을 통해 동시성의 거대한 숲과 Go 언어의 철학을 마스터하였으니,
이제 무기를 들고 전쟁터로 나갈 준비가 되었다.

[02-4] Go언어의 동시성 철학

(1) Go언어 공식 가이드가 주는 혼란

Go언어는 처음부터 CSP(채널) 이론을 뼈대에 품고 태어났다.
하지만 동시에 전통적인 방식인 ‘자물쇠(sync.Mutex)를 통한 메모리 동기화 도구’들도 완벽하게 지원한다.

이 두 가지 강력한 무기를 개발자 입맛대로 고를 수 있다는 건 엄청난 축복이지만, 초보자들에게는 큰 혼란을 주기도 한다.
왜냐하면 Go의 공식 문서나 핵심 개발자들의 인터뷰를 보면 오직 채널만이 정답인 것처럼 말할 때가 많기 때문이다.

- sync 패키지 공식 문서의 경고: “공유 자물쇠 같은 도구들은 저수준 라이브러리를 만들 때나 쓰는 것이다. 더 높은 차원의 동시성 동기화는 채널과 통신을 통해 처리하는 것이 훨씬 좋다.”

- Go 언어 공식 FAQ의 당부: “메모리를 공유함으로써 고루틴끼리 대화하려고 하지마라. 대신 대화(채널 통신)를 함으로써 메모리를 공유하라.”

이런 글들을 읽다 보면 “그럼 sync.Mutex 같은 자물쇠 도구들은 도대체 왜 만들어 둔 거지?” 하는 의문이 드는게 당연하다. 게다가 실무 코드를 보면 다들 자물쇠를 흔하게 쓰고 있다. 채널은 너무 남용하면 오히려 코드가 복잡해진다는 불평도 들린다.

Go 공식 위키에도 결국 이렇게 적혀있다.

“대부분의 동시성 문제는 채널로도 풀 수 있고, 전통적인 자물쇠로도 풀 수 있다. 둘 중 더 직관적이고 코드가 단순해지는 방식을 선택하세요.”

말은 참 좋지만, 초보자 입장에서는 여전히 모호하다.
“도대체 뭐가 더 단순하고 직관적인 건데?”라는 기준이 필요하다.

다행히 이를 명쾌하게 구별할 수 있는 나침반이 있다. 내가 지금 제어하려는 동시성의 범위가 ‘하나의 작은 클래스(구조체) 내부’에 갇혀 있는지, 아니면 ‘시스템 전체(외부 시스템 간의 흐름)’에 퍼져 있는지 확인하는 것이다.

(2) 채널 vs 자물쇠 선택을 위한 4가지 나침반

어떤 도구를 선택할지 고민될 때는 다음 4가지 질문을 스스로에게 던져보자.

질문1: 데이터의 ‘소유권’을 다른 코드로 넘겨주는 중인가?

- 무조건 채널

내가 가진 어떤 데이터나 계산 결과를 다른 고루틴에게 “이제 내가 가져가서 요리해봐!”하고 소유권을 토스하려고 한다면 채널이 완벽한 정답이다.

데이터에 명확한 주인을 정해주고, 한 번에 오직 하나의 고루틴만 그 데이터를 소유하게 만드는 것은 동시성 프로그램을 안전하게 만드는 최고의 방법이다.

채널은 코드 자체에 치러한 ‘소유권 이전의 의도’를 명확하게 각인시켜 준다.

- 부가이득1: 채널에 버퍼(Buffer)를 주면 메모리 내부에 아주 값싸고 훌륭한 대기열(Queue)을 똑딱 만들 수 있어서, 데이터를 만드는 사람과 소비하는 사람의 속도 차이를 분리할 수 있다.

- 부가이득2: 채널을 사용하면 내 코드가 다른 동시성 코드들과 레고 블럭처럼 부드럽게 결합된다.

질문2: 구조체 ‘내부의 비밀스러운 상태’를 보호하려는 것인가?

- 자물쇠

이 경우는 자물쇠가 채널을 압도하는 영역이다. 구조체를 설계할 때, 외부에서 이 구조체의 값을 바꿀 때 데이터가 오염되지 않도록 ‘내부 상태’만 숨겨서 보호하고 싶다면 전통적인 자물쇠 방식이 훨씬 깨끗하다.

자물쇠를 쓰면 함수를 호출하는 사람에게

“우리 내부에서 자물쇠를 잠그고 난리를 치고 있는 복잡한 비밀”을 완벽하게 숨겨줄 수 있다.

```
type Counter struct {
    mu   sync.Mutex // 내부 상태를 지킬 자물쇠
    value int
}

func (c *Counter) Increment() {
    c.mu.Lock()      // 함수 안에서만 조용히 잠그고
    defer c.mu.Unlock() // 끝날 때 열어줍니다.
    c.value++
}
```

외부 사용자는 이 구조체가 멀티스레드(고루틴) 환경에서 안전한지 어떤지 신경 쓸 필요 없이, 그냥 평범하게 Increment() 함수를 호출하기만 하면 된다.

- 주의할 점: 반드시 ‘내부’ 상태일 때만 자물쇠를 쓰자. 만약 자물쇠 자체를 구조체 밖으로 노출하거나, 다른 파일에서 자물쇠를 제어해야 하는 상황이 온다면 설계를 잘못했다는 강력한 경고 신호이다. 자물쇠는 언제나 아주 좁은 코드 범위 안에서만 숨겨져 있어야 한다.

질문3: 여러 개의 독립된 로직(시스템)을 서로 조율해야 하는가?

- 무조건 채널과 select 문

내 프로그램의 온갖 객체와 시스템 사방팔방에 자물쇠(Mutex)가 흩어져서 서로 잠그고 열고 지키고 있다고 상상해보자. 어떤 자물쇠가 언제 풀릴지 알 수 없어서 머지 않아 데드락(멈춤)이라는 끔찍한 악몽을 마주하게 될 것이다.

반면, 시스템 전체에 채널이 흐르고 있는 구조Go 언어에서 매우 권장되고 자연스러운 모습이다. 자물쇠는 결합할 수 없지만, 채널은 결합할 수 있다.

Go의 강력한 select 문 덕분에 채널들을 엮어서 거대한 시스템의 복잡성을 통제하기가 훨씬 쉬워진다. 만약 지금 자물쇠나 기본 도구를 쓰면서

“내 코드가 왜 데드락이 걸렸는지, 데이터 경쟁이 왜 나는지” 도저히 머리로 이해가 안가고 꼬여있다면, 당장 자물쇠를 다 버리고 채널 기반으로 코드를 리팩토링하라는 강력한 신호이다.

질문4: 극도로 성능이 중요한 ‘병목 구간’인가?

- 성능 측정을 먼저 해보고, 필요하다면 자물쇠(Mutex)를 고려하자.

이 말이 “내 프로그램은 무조건 빨라야 하니까 맨 처음부터 뮤텍스만 써야지!”라는 뜻은 절대 아니다.

기본적으로 채널도 내부적으로는 자물쇠 알고리즘을 사용해서 작동한다.

채널이라는 쓰기 편한 옷을 한 겹 더 입은 형태이기 때문에, 이론적으로 자물쇠보다 아주 미세하게 느릴 수 밖에 없다.

내 프로그램을 실제로 정밀하게 테스트해 보았더니, 특정 구간이 전체 시스템 속도를 엄청나게 잡아먹는 극심한 병목 구간으로 판명되었고, 채널의 미세한 오버헤드조차 견어내야 하는 극한의 상황이라면, 그때 비로소 원초적인 자물쇠를 사용해 성능을 쥐어 짜내는 것이 맞다. 하지만 이 단계로 가기전에, 애초에 프로그램 구조 전체를 더 효율적으로 뜯어고칠 순 없는지 먼저 고민해 보는 것이 좋다.

(3) 기존의 구시대적 패턴 버리기

과거 운영체제의 무거운 ‘OS 스레드’를 기반으로 발전해 온 자바나 C++같은 언어들에서는 스레드 생성 비용이 너무 비싸서 ‘스레드 풀(Thread Pool/Worker Pool)’같은 복잡한 관리 패턴들을 필수로 구현해야 했다.

하지만 Go 언어에서는 과감하게 이러한 옛날 패턴들을 머릿속에서 지워버려도 좋다. Go의 고루틴은 하드웨어가 비명을 지르기 전까지 수만 개, 수십만 개를 켜도 될 정도로 상상 초월하게 가볍다. 고루틴 개수 제한을 걸어두는 코드를 짜느라 아까운 시간을 낭비하는 것보다, 내 업무 흐름의 동시성 요구사항 그대로 고루틴을 자유롭게 아낌없이 켜는 것이 훨씬 Go 다운 방식이다. 하드웨어 한계에 부딪히기 전에, 프로그램 구조를 맑게 유지하는 것이 훨씬 중요하다.

(4) 결론

Go 언어의 위대한 동시성 철학은 결국 다음 세 문장으로 완벽하게 요약할 수 있다.

- 복잡하게 꼬지 말고, 언제나 단순함을 최우선 목표로 삼아라.
- 가능한 모든 곳에 채널을 우선적으로 사용해라
- 고루틴을 만들 때는 돈이 들지 않는 공짜 자원처럼 아낌없이 써라

이 3가지 기준을 마음에 품고 코드를 작성한다면, 여러분도 머지 않아 버그 없고 우아한 Go 동시성 프로그램의 마스터가 될 수 있을 것이다.